

Informe de Desarrollo de Software – febrero 2026

Atuntaqui, 13 de marzo de 2026

Desarrollo de software web – backend

El siguiente documento detalla las actividades técnicas realizadas para la implementación de la arquitectura limpia y los estándares de desarrollo en el proyecto EPAA-AA, correspondientes al periodo de febrero de 2026. Basado en los paradigmas de Clean Architecture y los principios SOLID, este informe certifica la robustez, testeabilidad y escalabilidad del software, garantizando un mantenimiento sostenible y libre de efectos colaterales.

1. Visión Arquitectónica: Clean Architecture en EPAA-AA

El proyecto EPAA-AA es una plataforma web de vanguardia diseñada para la gestión integral de servicios de Agua Potable y Alcantarillado, optimizando tanto los procesos logísticos como la experiencia final del usuario. El pilar fundamental de su diseño es la independencia: el software se desacopla de frameworks, bases de datos e interfaces externas. La dirección de las dependencias fluye estrictamente hacia el Dominio, salvaguardando la lógica de negocio frente a la evolución tecnológica.

La arquitectura limpia es una metodología estratégica que segmenta las responsabilidades del sistema en capas aisladas. Esta separación de preocupaciones facilita la evolución del software, la ejecución de pruebas unitarias y el crecimiento del sistema. En EPAA-AA, este estándar se aplica uniformemente en todos los módulos, asegurando integridad estructural y excelencia técnica.

1.1 Estructura de Capas (Jerarquía)

La solución se organiza en módulos funcionales clave, tales como:

- *accounting* (Contabilidad)
- *trash* (Gestión de Tasa de Basura)
- *readings* (Lecturas)
- *users* (Usuarios)
- *reports* (Reportes)
- *settings* (Configuración)
- *dashboard* (Indicadores)

Cada uno de estos módulos (ej: *accounting*, *trash*, *readings*, *users*, *reports*, *settings*, *dashboard*) se estructura en cuatro capas concéntricas de especialización:

- 1. Capa de Dominio (Domain) - El Corazón: La capa más estable y pura. Define las reglas de negocio esenciales, contratos (interfaces) y modelos de datos fundamentales, permaneciendo ajena a cualquier detalle de implementación externa.
- 2. Capa de Aplicación (Application) - La Orquestación: Responsable de implementar los Casos de Uso. Esta capa dirige el flujo de información hacia y desde el dominio, transformando los requisitos de negocio en acciones de software coordinadas.

- 3. Capa de Infraestructura (Infrastructure) - El Detalle Técnico: Provee la implementación concreta de los contratos del dominio. Gestiona la comunicación con APIs, bases de datos y la integración de herramientas externas (como *jspdf* o *exceljs*).
- 4. Capa de Presentación (Presentation) - La Interfaz: Gestiona la interacción con el usuario final a través de componentes React y Hooks (ViewModels), encargándose de la renderización visual y el manejo del estado de la UI.

2. Guía de Desarrollo de un Módulo (Proceso Estándar)

Para mantener la coherencia arquitectónica, todo nuevo desarrollo debe ejecutar el siguiente flujo de trabajo:

Fase A: Definición del Dominio (Domain)

1. Modelado: Creación de interfaces TypeScript en *domain/models/*.
2. Contrato: Definición de interfaces de repositorio en *domain/repositories/*.

La capa de dominio representa la esencia del sistema. Alberga las reglas críticas del negocio y los modelos de datos. Al ser la capa más importante, su pureza es vital: no debe poseer dependencias de ninguna otra capa, asegurando que el negocio no se vea afectado por cambios en la tecnología.

Fase B: Implementación Técnica (Infrastructure)

1. Repositorio: Implementación de clases que consumen la API en *infrastructure/repositories/*.
2. Mapping: Lógica de transformación para convertir datos de sistemas Legados (JSON) en modelos limpios del Dominio.

La capa de infraestructura actúa como el puente con el mundo exterior. Es el lugar donde reside el conocimiento sobre tecnologías específicas (APIs, bases de datos). Al concentrar aquí las dependencias externas, protegemos al resto de la aplicación de su complejidad.

Fase C: Lógica de Aplicación (Application)

1. Caso de Uso: Implementación de clases *Execute[Action]UseCase* en *application/usecases/*.
2. Regla: Cada caso de uso debe ser atómico, representando una única acción de negocio mediante su método público *execute*.

La capa de aplicación orquesta la interacción entre el mundo exterior y el dominio puro. A diferencia de la infraestructura, esta capa debe permanecer independiente de tecnologías externas, dependiendo únicamente de las abstracciones del dominio para cumplir con la lógica de la aplicación.

Fase D: Presentación (Presentation)

1. ViewModel (Hook): Gestión de estados de carga (*isLoading*), errores y consumo de Casos de Uso.
2. Componentes: Construcción de UI en *presentation/components/*.
3. Estilos: Implementación de CSS modular y consistente.

Esta capa es el punto de contacto con el usuario. Utiliza React y patrones de ViewModels para desacoplar la lógica visual del estado de la aplicación. Aunque depende de la librería de UI, mantiene su lógica de negocio delegada en la capa de aplicación.

3. Implementación Práctica: Ejemplo Real (Módulo Trash)

Ejecución del flujo de datos completo, desde el contrato base hasta la visualización final.

3.1 Dominio: Contratos y Modelos

En esta fase inicial, definimos rigurosamente la estructura de datos requerida por el negocio y los contratos que la infraestructura estará obligada a cumplir. Es un entorno libre de frameworks, compuesto exclusivamente por interfaces puras.

```
// Modelo: domain/models/trash-rate-report.model.ts
export interface TrashRateKPI {
  collectorId: string;
  totalCollected: number;
  paymentCount: number;
}
// Contrato: domain/repositories/interface.repository.ts
export interface InterfaceTrashRateReportRepository {
  getTrashRateKPI(params: DateRangeParams): Promise<TrashRateKPI[]>;
}
```

3.2 Infraestructura: Implementación del Repositorio

Implementamos la lógica técnica necesaria para consumir la API en *infrastructure/repositories/*. Esta capa es la encargada de manejar la especificidad del transporte y la serialización de datos externos, operando bajo las interfaces definidas por el dominio.

```
export class TrashRateRepositoryImpl implements InterfaceTrashRateReportRepository {
  constructor(private readonly client: HttpClientInterface) {}

  async getTrashRateKPI(params: DateRangeParams): Promise<TrashRateKPI[]> {
    const response = await this.client.get<ApiResponse<TrashRateKPI[]>>(
      '/trash-rate-report/trash-rate-kpi',
      { params }
    );
    return response.data.data; // Transformación implícita al modelo de dominio
  }
}
```

3.3 Aplicación: El Caso de Uso

Orquestamos la acción mediante clases *Execute[Action]UseCase*. Esta capa garantiza que el flujo de datos respete las reglas de la aplicación, interactuando con las abstracciones del repositorio sin conocer los detalles de la infraestructura.

```
export class GetTrashRateKPIUseCase {
  constructor(
    private readonly repository: InterfaceTrashRateReportRepository
  ) {}

  async execute(params: DateRangeParams): Promise<TrashRateKPI[]> {
    // Aquí se podrían incluir validaciones o lógica de orquestación compleja
    return await this.repository.getTrashRateKPI(params);
  }
}
```

```

}
}

```

3.4 Presentación: ViewModel (Hook)

Gestionamos el ciclo de vida visual de la funcionalidad: estados de carga, manejo de errores y suscripción a los datos. El ViewModel separa la complejidad del estado de la UI de la lógica de negocio pura.

```

export const useTrashRateKPIViewModel = () => {
  const [data, setData] = useState<TrashRateKPI[]>([]);
  const [loading, setLoading] = useState(false);

  const handleFetch = async (params: DateRangeParams) => {
    setLoading(true);
    try {
      const result = await getTrashRateKPIUseCase.execute(params);
      setData(result);
    } catch (error) {
      // Manejo centralizado de errores de presentación
    } finally {
      setLoading(false);
    }
  };
  return { data, loading, handleFetch };
};

```

4. Principios SOLID Aplicados

Principio	Aplicación Técnica en EPAA-AA	Caso de Uso Real
S - Responsabilidad Única	Cada clase o componente tiene una sola razón para cambiar.	El componente <i>PaymentsTable</i> solo renderiza; no sabe cómo se generan los PDFs.
O - Abierto/Cerrado	Software abierto a extensión, pero cerrado a modificación.	<i>ExportService</i> puede añadir soporte para CSV sin cambiar el código de los módulos.
L - Sustitución de Liskov	Las subclases deben ser sustituibles por sus clases base.	<i>FetchClient</i> y <i>AxiosClient</i> son intercambiables mediante <i>HttpClientInterface</i> .
I - Segregación de Interfaz	No obligar a depender de interfaces que no se usan.	Las props de <i>Button</i> solo exponen atributos visuales y de evento estrictamente necesarios.
D - Inversión de Dependencias	Depender de abstracciones, no de concreciones.	Los hooks reciben abstracciones (Casos de Uso) vía contextos o inyección lógica.

5. Estándares de Calidad y UI/UX

5.1 Ingeniería de Código

- Tipado Estricto: Eliminación total del uso de *any*. Empleo de interfaces claras y genéricos para todas las respuestas de API.
- Inyección de Dependencias: Prohibición de instanciar implementaciones técnicas directamente dentro de los componentes de UI.
- Inmutabilidad: Uso riguroso de estados inmutables en React para garantizar la predictibilidad del sistema.

5.2 Experiencia de Usuario (UX)

- Localización (i18n): Eliminación de strings literales en el código. Toda la interfaz se gestiona mediante el motor de traducción *useTranslation*.
- Diseño Adaptativo y Tematización: Soporte nativo para Dark Mode mediante variables de diseño y tokens de CSS.
- Feedback Visual Proactivo: Implementación sistemática de estados de carga (*CircularProgress*) y vistas optimizadas para datos vacíos.

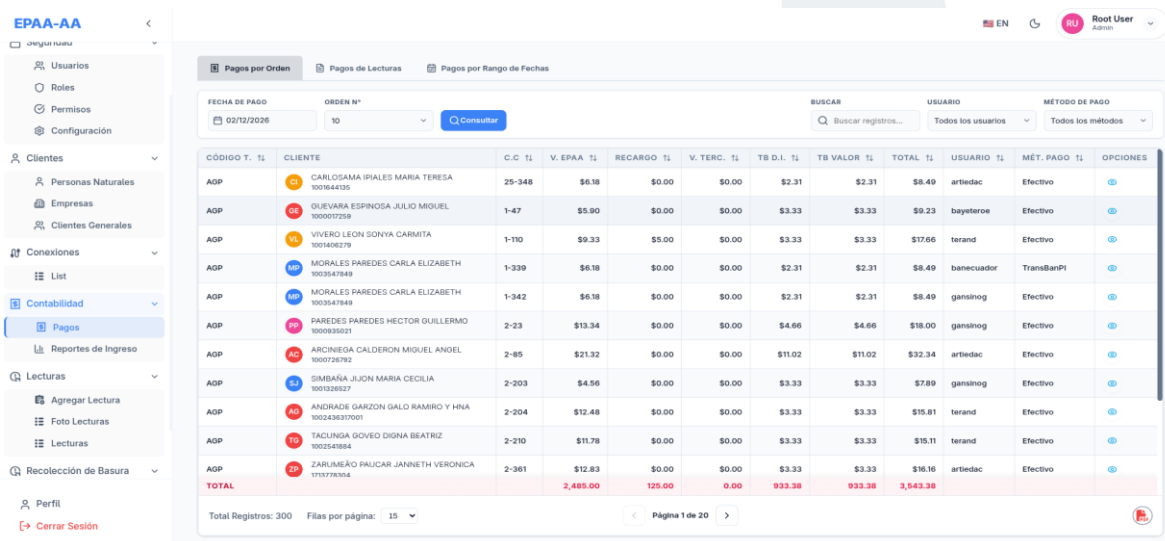
5.3 Caso de Estudio: Arquitectura de Exportación (PDF)

1. Componente: Invoca la acción de exportación delegando la lógica.
2. Hook (*useTablePdfExport*): Actúa como orquestador, formateando la data y gestionando la interfaz del modal.
3. Servicio (*ExportService*): Capa de infraestructura que encapsula la complejidad de *jspdf-autotable*.

6. Evidencias y Resultados de la Implementación

A continuación, se presenta la validación visual y técnica de la arquitectura implementada en los módulos críticos durante el periodo de febrero de 2026. Estas evidencias demuestran la cohesión visual, la integridad de los datos y el cumplimiento de los estándares de ingeniería definidos.

Módulo de Contabilidad (Gestión de Pagos)

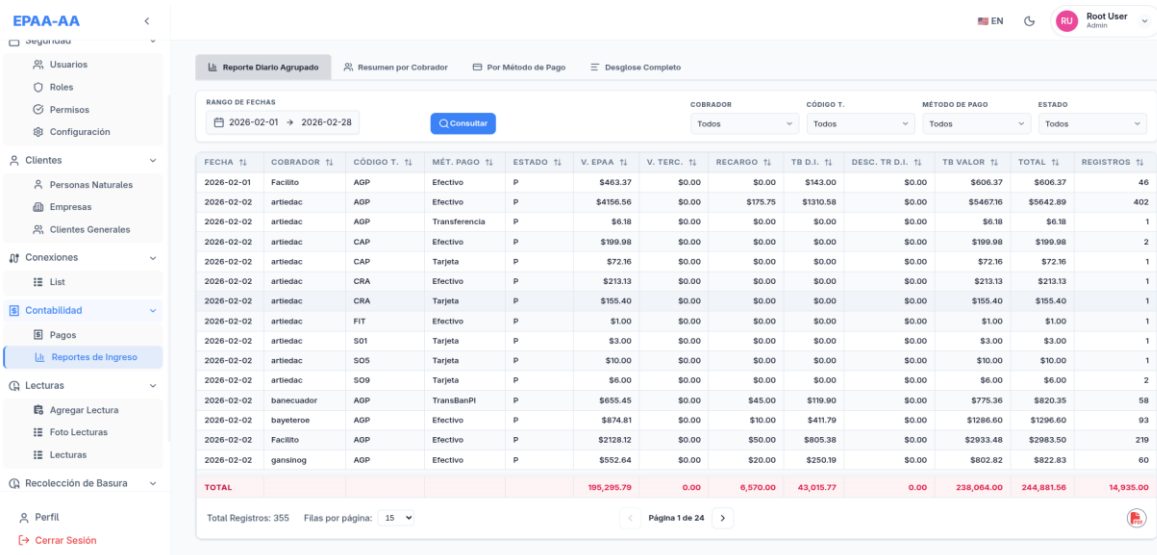


CÓDIGO T. TI	CLIENTE	C.C TI	V. EPAA TI	RECARGO TI	V. TERC. TI	TB D.I. TI	TB VALOR TI	TOTAL TI	USUARIO TI	MÉT. PAGO TI	OPCIONES
AGP	CARLODAMA IPALES MARIA TERESA 901644105	25-348	\$6.18	\$0.00	\$0.00	\$2.31	\$2.31	\$8.49	artiedac	Efectivo	
AGP	OLIVERA ESPINOSA JULIO MIGUEL 900007259	1-47	\$5.90	\$0.00	\$0.00	\$3.33	\$3.33	\$9.23	bayeteros	Efectivo	
AGP	VIVERO LEON SONYA CARMITA 901406279	1-110	\$9.33	\$5.00	\$0.00	\$3.33	\$3.33	\$17.66	terand	Efectivo	
AGP	MORALES PAREDES CARLA ELIZABETH 903547849	1-339	\$6.18	\$0.00	\$0.00	\$2.31	\$2.31	\$8.49	banecuator	TransBanPI	
AGP	MORALES PAREDES CARLA ELIZABETH 903547849	1-342	\$6.18	\$0.00	\$0.00	\$2.31	\$2.31	\$8.49	gansinog	Efectivo	
AGP	PARDES PAREDES HECTOR GUILLERMO 900938821	2-23	\$13.34	\$0.00	\$0.00	\$4.66	\$4.66	\$18.00	gansinog	Efectivo	
AGP	ARCINIEGA CALDERON MIGUEL ANDEL 900726782	2-85	\$21.32	\$0.00	\$0.00	\$11.02	\$11.02	\$32.34	artiedac	Efectivo	
AGP	SIMBAÑA JIJON MARIA CECILIA 901328827	2-203	\$4.56	\$0.00	\$0.00	\$3.33	\$3.33	\$7.89	gansinog	Efectivo	
AGP	ANDRADE GARZON GALO RAMIRO Y HNA 90243637001	2-204	\$12.48	\$0.00	\$0.00	\$3.33	\$3.33	\$15.81	terand	Efectivo	
AGP	TACLINGA GIOVEO DIGNA BEATRIZ 902242864	2-210	\$11.78	\$0.00	\$0.00	\$3.33	\$3.33	\$15.11	terand	Efectivo	
AGP	ZARUMERO RAICAR JANNETH VERONICA 913778354	2-961	\$12.83	\$0.00	\$0.00	\$3.33	\$3.33	\$16.16	artiedac	Efectivo	
TOTAL			2,485.00	125.00	0.00	933.38	933.38	3,543.38			

Total Registros: 300 Filas por página: 15

Descripción: Interfaz de gestión de pagos por orden. Se observa la implementación de filtros dinámicos (fecha, orden, usuario, método de pago) y una tabla de datos tipada que garantiza la integridad de la información financiera. El diseño sigue el estándar de la capa de Presentación con feedback visual claro.

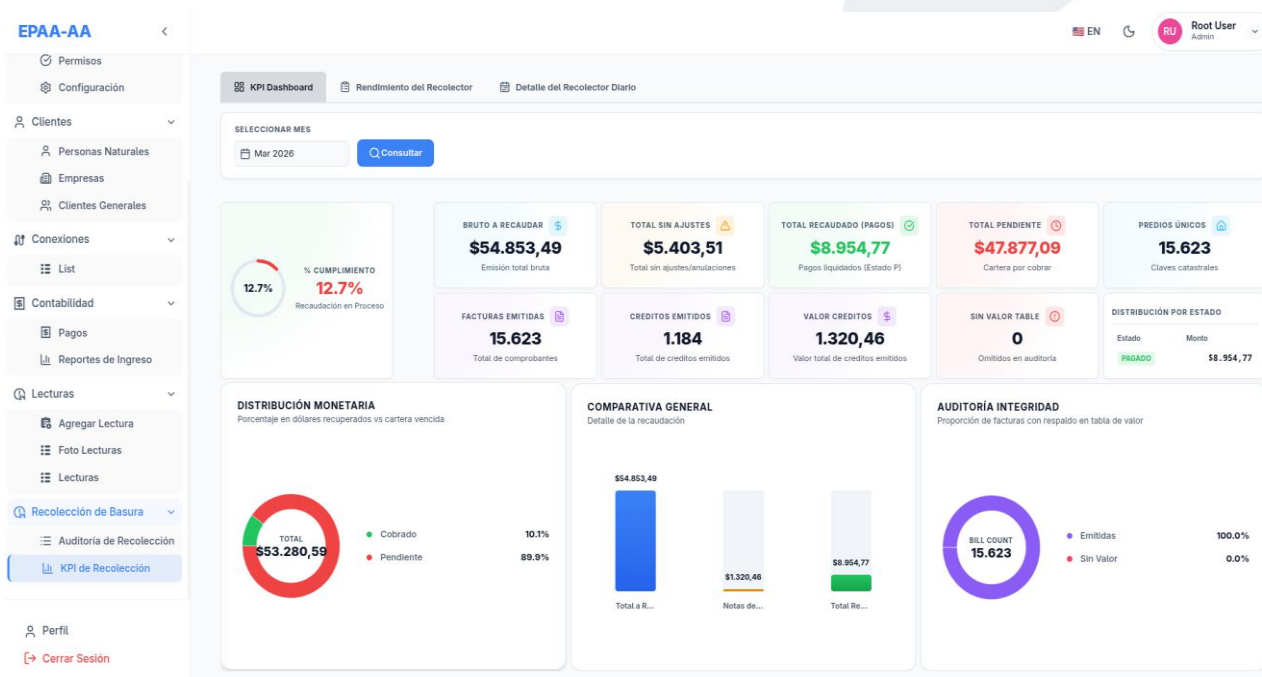
Reportes de Ingresos Consolidados



FECHA	COBRADOR	CÓDIGO T.	MÉT. PAGO	ESTADO	V. EPAA	V. TERC.	RECARGO	TB D.I.	DESC. TR D.I.	TB VALOR	TOTAL	REGISTROS
2026-02-01	Facilito	AGP	Efectivo	P	\$463.37	\$0.00	\$143.00	\$0.00	\$0.00	\$606.37	\$606.37	46
2026-02-02	artiedac	AGP	Efectivo	P	\$4156.56	\$0.00	\$175.75	\$1310.58	\$0.00	\$5467.16	\$5642.89	402
2026-02-02	artiedac	AGP	Transferencia	P	\$6.18	\$0.00	\$0.00	\$0.00	\$0.00	\$6.18	\$6.18	1
2026-02-02	artiedac	CAP	Efectivo	P	\$199.98	\$0.00	\$0.00	\$0.00	\$0.00	\$199.98	\$199.98	2
2026-02-02	artiedac	CAP	Tarjeta	P	\$72.16	\$0.00	\$0.00	\$0.00	\$0.00	\$72.16	\$72.16	1
2026-02-02	artiedac	CRA	Efectivo	P	\$213.13	\$0.00	\$0.00	\$0.00	\$0.00	\$213.13	\$213.13	1
2026-02-02	artiedac	CRA	Tarjeta	P	\$155.40	\$0.00	\$0.00	\$0.00	\$0.00	\$155.40	\$155.40	1
2026-02-02	artiedac	FIT	Efectivo	P	\$1.00	\$0.00	\$0.00	\$0.00	\$0.00	\$1.00	\$1.00	1
2026-02-02	artiedac	SO1	Tarjeta	P	\$3.00	\$0.00	\$0.00	\$0.00	\$0.00	\$3.00	\$3.00	1
2026-02-02	artiedac	SO5	Tarjeta	P	\$10.00	\$0.00	\$0.00	\$0.00	\$0.00	\$10.00	\$10.00	1
2026-02-02	artiedac	SO9	Tarjeta	P	\$6.00	\$0.00	\$0.00	\$0.00	\$0.00	\$6.00	\$6.00	2
2026-02-02	banecuator	AGP	TransBanPI	P	\$655.45	\$0.00	\$45.00	\$119.90	\$0.00	\$775.36	\$820.35	58
2026-02-02	bayerroee	AGP	Efectivo	P	\$874.81	\$0.00	\$10.00	\$411.79	\$0.00	\$1286.60	\$1296.60	93
2026-02-02	Facilito	AGP	Efectivo	P	\$2118.12	\$0.00	\$50.00	\$805.38	\$0.00	\$2833.48	\$2983.50	219
2026-02-02	gansinog	AGP	Efectivo	P	\$552.64	\$0.00	\$20.00	\$250.19	\$0.00	\$802.82	\$822.83	60
TOTAL					195,295.79	0.00	6,570.00	43,015.77	0.00	238,064.00	244,881.56	14,935.00

Descripción: Vista de reportes de ingresos agrupados. Esta funcionalidad utiliza la arquitectura de exportación para generar documentos PDF profesionales. La orquestación entre el ViewModel y el ExportService permite que el usuario obtenga informes detallados por recaudador y método de pago con alta precisión.

Módulo de Tasa de Basura (KPI Dashboard)



Descripción: Panel de indicadores clave (KPI) para la recolección de basura. Implementa el patrón de carga proactiva de datos mediante ViewModels, mostrando métricas críticas como recaudación bruta, facturación emitida y porcentajes de cumplimiento.

Rendimiento y Auditoría de Recolección

EPAA-AA

- Permisos
- Configuración
- Cientes
 - Personas Naturales
 - Empresas
 - Cientes Generales
- Conexiones
 - List
- Contabilidad
 - Pagos
 - Reportes de Ingreso
- Lecturas
 - Agregar Lectura
 - Foto Lecturas
 - Lecturas
- Recolección de Basura
 - Auditoría de Recolección
 - KPI de Recolección

Perfil
Cerrar Sesión

EN
RU Root User Admin

KPI Dashboard
Rendimiento del Recolector
Detalle del Recolector Diario

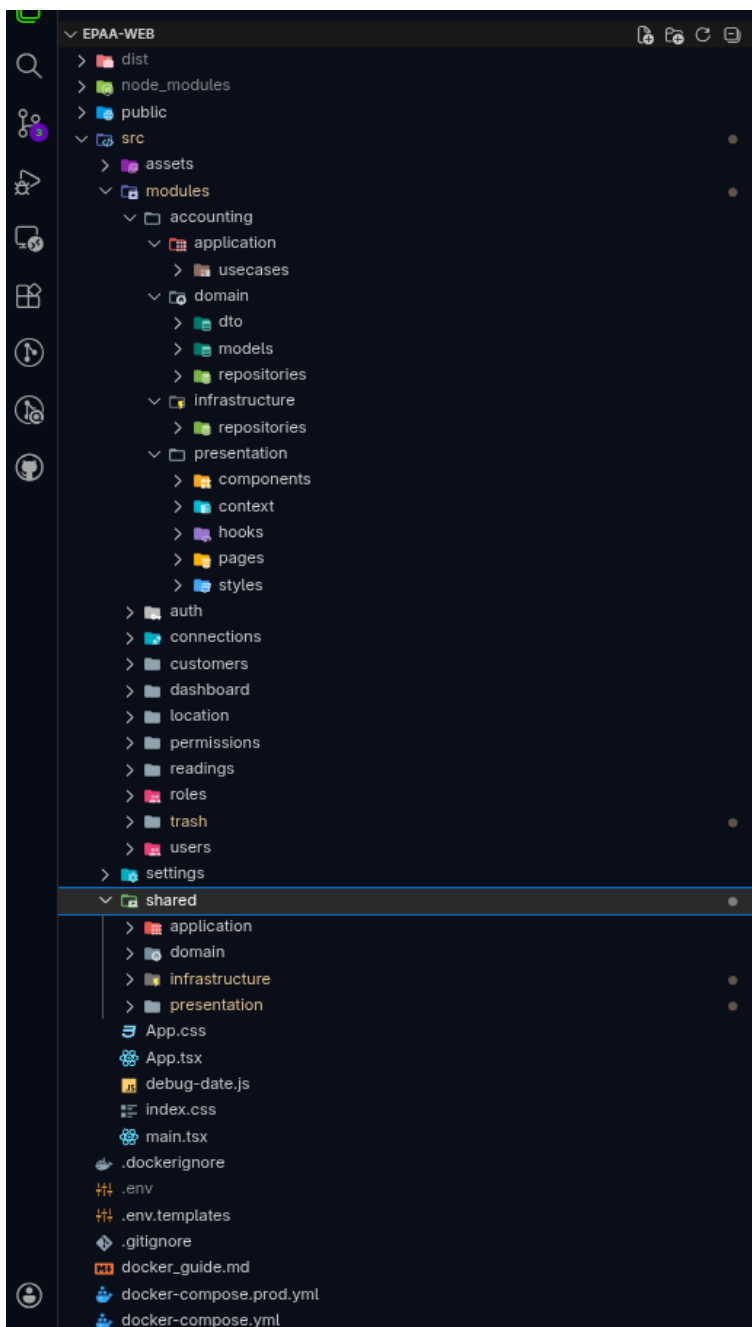
RANGO DE FECHAS
2026-03-01 → 2026-03-31

ID DEL COBRADOR	N° DE FACTURAS	TB DATOS INGRESO	TB TABLA VALOR	DIFERENCIA (TBDI - TBTV)	MONTO FACTURADO	DESC. APLICADOS	RECAUDACIÓN NETA	VALOR FACT. (A - B)	FACT. (C)
terand	966	3,052.24	3,052.24	0.00	3,052.24	591.48	2,460.76	0.00	
artiedac	987	2,823.90	2,823.90	0.00	2,823.90	527.49	2,296.41	0.00	
Facilito	557	1,785.10	1,780.73	-4.37	1,785.10	298.07	1,487.03	0.00	
banecuador	172	476.96	476.96	0.00	476.96	23.96	453.00	0.00	
gansinog	152	460.52	460.52	0.00	460.52	83.56	376.96	0.00	
bayeteroe	126	356.05	356.05	0.00	356.05	48.34	307.72	0.00	
TOTAL	2,960.00	8,954.77	8,950.40	-4.37	8,954.77	1,572.90	7,381.87	0.00	

Total Registros: 6
Filas por página: 15

Descripción: Detalle de auditoría y rendimiento por recolector. Esta vista demuestra la capacidad del sistema para procesar grandes volúmenes de datos y presentarlos de forma analítica, facilitando la toma de decisiones gerenciales sobre la eficiencia operativa.

Arquitectura de Capas (Estructura de Directorios)



Descripción: Evidencia de la organización del código fuente en VS Code. La jerarquía src/modules/[module] (Domain, Application, Infrastructure, Presentation) confirma el cumplimiento estricto de Clean Architecture, asegurando que cada componente tenga una responsabilidad única y un lugar definido.

Nota: Preservación de la Arquitectura, Para mantener la integridad del sistema, cualquier cambio que vulnere la dirección de las dependencias (ej: que el Dominio importe componentes de Presentación) debe ser desestimado en las revisiones técnicas.

ANEXOS

EPAA-AA

- Panel Principal
- Reportes
- Seguridad
- Cientes
- Conexiones
- Contabilidad
- Pagos
- Reportes de Ingreso
- Lecturas
- Agregar Lectura
- Foto Lecturas
- Lecturas
- Recolección de Basura
- Auditoría de Recolección
- KPI de Recolección

Perfil
Cerrar Sesión

Registro de Lecturas

1713569828 - SANTIAGO RAFAEL MORA ANDRADE

12-36

Consumo Prom. **10.40 m³**
Promedio de los últimos 10 periodos

Consumo Anterior **10.00 m³**
Fecha: 11/3/2026

Consumo Actual **0.00 m³**
Fecha: 13/3/2026

Lectura Anterior 2026-03

206.00

Lectura Actual (Obligatorio)

216.00

Descripción o Novedades (Opcional)

Ingrese una descripción detallada...

Historial de Lecturas Recientes

MES/AÑO	FECHA LECT.	HORA LECT.	LECTURA ANT.	LECTURA ACT.	CONSUMO (m³)	OBSERVACIÓN
MARZO / 2026	11/3/2026	4:43:27 a. m.	206.00	216.00	10.00	NORMAL
FEBRERO / 2026	2/2/2026	10:27:33 a. m.	189.00	206.00	17.00	CONSUMO ALTO
ENERO / 2026	8/1/2026	12:48:00 a. m.	164.00	189.00	25.00	CONSUMO EXCESIVO
DICIEMBRE / 2025	9/12/2025	7:00:00 p. m.	144.00	164.00	20.00	NORMAL
NOVIEMBRE / 2025	23/11/2025	7:00:00 p. m.	138.00	144.00	6.00	NORMAL
OCTUBRE / 2025	16/10/2025	7:00:00 p. m.	132.00	138.00	6.00	NORMAL
SEPTIEMBRE / 2025	24/9/2025	7:00:00 p. m.	126.00	132.00	6.00	NORMAL

Total Registros: 15 Filas por página: 5

Página 1 de 3

Ver Información Adicional

EPAA-AA

- Panel Principal
- Reportes
- Seguridad
- Cientes
- Conexiones
- Contabilidad
- Pagos
- Reportes de Ingreso
- Lecturas
- Agregar Lectura
- Foto Lecturas
- Lecturas
- Recolección de Basura
- Auditoría de Recolección
- KPI de Recolección

Perfil
Cerrar Sesión

Registro Fotográfico de Lecturas

Visualiza las evidencias fotográficas capturadas en campo de manera rápida y profesional.

MODO DE BÚSQUEDA CLAVE CATASTRAL EXACTA SECTOR (OPCIONAL)
Mes y Sector Mar 2026 Todos los sectores Consultar

CLAVE CATASTRAL	MES	AÑO	LECT. ANTERIOR	LECT. ACTUAL	CONSUMO	NOVEDAD	IMÁGENES
11-30	MARZO	2026	0.00	0.00	0.00 m³	CONSUMO MUY BAJO	Ver (2)
11-301	MARZO	2026	111.00	169.00	58.00 m³	CONSUMO EXCESIVO	Ver (2)
11-367	MARZO	2026	2999.00	2999.00	0.00 m³	CONSUMO MUY BAJO	Ver (1)
11-380	MARZO	2026	3274.00	3315.00	41.00 m³	NORMAL	Ver (1)
1-213	MARZO	2026	4699.00	4805.00	106.00 m³	CONSUMO EXCESIVO	Ver (2)
17-248	MARZO	2026	4740.00	4802.00	62.00 m³	NORMAL	Ver (1)
17-304	MARZO	2026	2884.00	2903.00	19.00 m³	CONSUMO EXCESIVO	Ver (1)
19-104	MARZO	2026	1922.00	1945.00	23.00 m³	NORMAL	Ver (1)
19-115	MARZO	2026	74.00	136.00	62.00 m³	NORMAL	Ver (1)
19-166	MARZO	2026	1923.00	1970.00	47.00 m³	CONSUMO EXCESIVO	Ver (1)

Total Registros: 82 Filas por página: 10

Página 1 de 9

Mario Salazar
DESARROLLADOR DE SOFTWARE